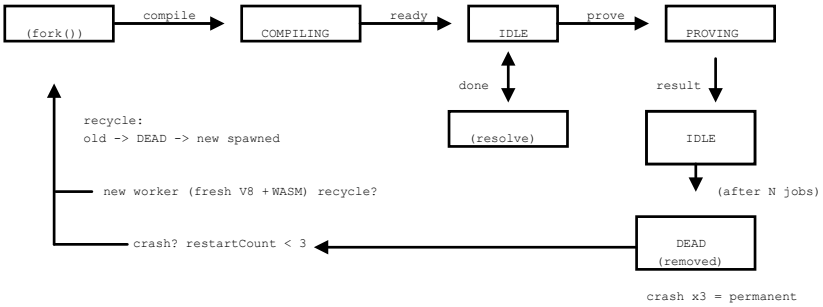
FIG. 16B — PROVING POOL — WORKER LIFECYCLE, INTEGRATION & WASM RECYCLING 1606

SECTION D: WORKER LIFECYCLE AND PROVING PIPELINE 1606

WORKER STATE MACHINE:

```
WorkerState = 'compiling' | 'idle' | 'proving' | 'dead'
```

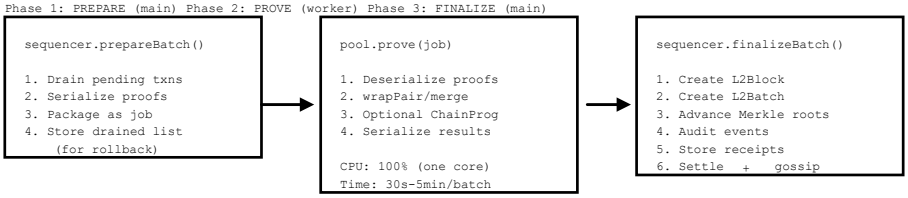



```
Compilation (sequential, dependency order):
  Step 1: StateTransitionProgram.compile() (BatchProgram references its VK)
  Step 2: BatchProgram.compile() (wrapSingle, wrapPair, mergeBatches)
  Step 3: ChainProgram.compile() (genesis, advance — references BatchProof VK)
  Total: ~2-5 min/worker. Workers compile ONE AT A TIME to avoid CPU thrashing.

handleProve pipeline:
  1. DESERIALIZE: StateTransitionProof.fromJSON(json) for each transition
  2. BUILD BatchProof: 1 proof? wrapSingle(p,p)
                       2+ proofs? Layer 1: wrapPair(p[0],p[1]), wrapPair(p[2],p[3])...
                                  Layer 2 + : mergeBatches(pair[0],pair[1])... (binary tree -> 1)
      Example: p0 p1 p2 p3 --> pair0 pair1 --> batch (wrapPair then mergeBatches)
  3. BUILD ChainProof: if adminState: genesis(batch,...) or advance(prevChain,batch,...)
  4. SERIALIZE: batchProof.toJSON() + chainProof?.toJSON() -> IPC send
```

SECTION E: SEQUENCER INTEGRATION (THREE-PHASE PIPELINE) 1608

L2Node.executeBatchWithPool():

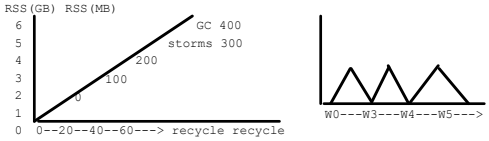



```
On failure: sequencer.restoreDrainedTransitions() — return to pending pool, no state mutation.
Only Phase 2 (CPU-intensive) moves to worker. State mutations NEVER cross IPC boundary.
With N workers: N batches prove in parallel. Main process stays responsive.
```

SECTION F: WORKER RECYCLING (WASM HEAP RECLAMATION) 1610

```
PROBLEM: o1js WASM heap grows ~60MB/proof. Pickles has NO compaction API. After 64 + proofs:
  RSS 5 + GB -> V8 GC storms -> event loop stall.

  RSS (no recycling): RSS (recycleAfterJobs=5):
```




```
  recycleAfterJobs=N: After N prove jobs, kill worker -> spawn fresh -> recompile.
  New worker = new V8 isolate + new WASM heap = back to baseline RSS.

  recycleWorker(handle) procedure:
    1. Emit 'worker:recycled' event              5. spawnWorker() — new child process
    2. Set status='dead', DELETE                    (fresh V8 isolate, fresh WASM heap)
       from workers Map                          6. Send 'compile' -> wait 'ready' (2-5 min)
    3. Send 'shutdown' IPC to old worker         7. drainQueue() — dispatch queued jobs
    4. setTimeout(graceMs) ->
       still alive? -> SIGTERM
       (NOT crash — restartCount NOT incremented)

  RECYCLE (planned, after N jobs):                    CRASH (unplanned, process exit):
    Graceful shutdown, no restartCount incr,            Abrupt, restartCount++,
    always respawns, no job failure                     respawns if count<3, in-flight job REJECTED
```